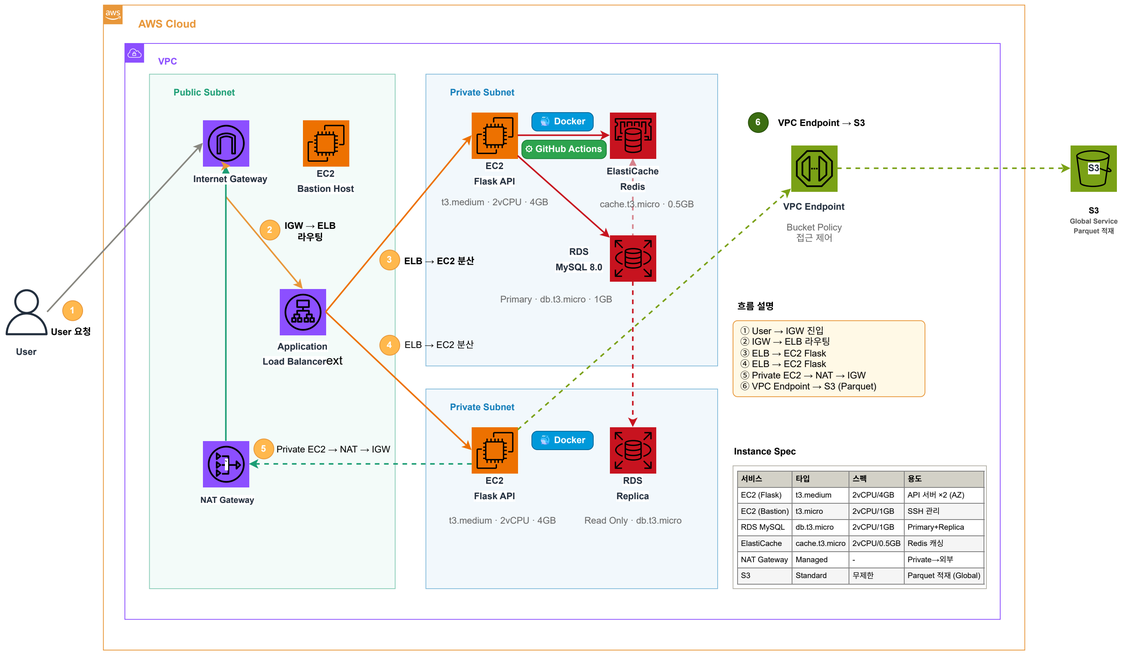

흐름 설명

① User → IGW 진입
② IGW → ELB 라우팅
③ ELB → EC2 Flask
④ ELB → EC2 Flask
⑤ Private EC2 → NAT → IGW
⑥ VPC Endpoint → S3 (Parquet)

Instance Spec

서비스	타입	스펙	용도
EC2 (Flask)	t3.medium	2vCPU/4GB	API 서버 ×2 (AZ)
EC2 (Bastion)	t3.micro	2vCPU/1GB	SSH 관리
RDS MySQL	db.t3.micro	2vCPU/1GB	Primary+Replica
ElastiCache	cache.t3.micro	2vCPU/0.5GB	Redis 캐싱
NAT Gateway	Managed	-	Private→외부
S3	Standard	무제한	Parquet 적재 (Global)

번호	라벨	설명
1	User 요청	사용자가 브라우저에서 Steam ID 또는 URL을 입력하면 HTTP/HTTPS 요청이 발생 -> 이 요청은 공개 인터넷을 타고 AWS의 **Internet Gateway(IGW)** 로 진입 IGW는 VPC와 인터넷 사이의 관문 역할을 하는 컴포넌트 인터넷 → IGW로 트래픽 전달
2	IGW → ELB 라우팅	IGW를 통과한 트래픽이 **Application Load Balancer(ELB)** 로 전달. Application Load Balancer 수신
3	ELB → EC2 분산	ELB가 트래픽을 **Private Subnet AZ-1에 있는 EC2** 로 보냄. 이 EC2 안에는 **Flask REST API 서버**가 Docker 컨테이너로 실행. 사용자 요청을 받아서 게임 추천 로직을 처리하고 JSON 형태로 응답을 리턴. GitHub Actions로 CI/CD 파이프라인이 구성되어 있어서 코드가 GitHub에 push되면 자동으로 Docker 이미지를 빌드하고 이 EC2에 배포. **RDS MySQL** 에는 사용자 정보와 게임 데이터가 저장 Flask REST API · Docker · GitHub Actions CI/CD
4	ELB → EC2 분산	③과 동일한 역할을 하는 EC2가 **다른 가용영역(AZ-2)** 에도 존재 가용영역 이중화 · 고가용성 확보
5	Private EC2 → NAT → IGW (외부 출구)	**Snowflake 적재** 정제된 데이터를 Snowflake Endpoint로 Load, ELT 구조이므로 Load 먼저, Transform은 Snowflake 안에서 수행 외부 인터넷 출구 (패키지 설치, 외부 API 호출)
6	VPC Endpoint → S3 (Parquet 적재)	Parquet 적재 · Bucket Policy 접근제어 (인터넷 우회 없이 내부 연결) **Bucket Policy**로 특정 VPC에서만 이 버킷에 접근할 수 있도록 제한을 걸어서 보안을 강화